
ENUNCIADO ACADÉMICO DEL PROYECTO INTEGRADOR

UMBRAL

*Plataforma para la operación en tiempo real de experiencias de investigación
inmersiva*

Programa	Proyecto integrador de desarrollo de software
Proyecto	UMBRAL
Modalidad	Trabajo por equipos con entregas parciales y presentación final
Tecnologías base	React, WebSockets, .NET Core, Entity Framework Core, MediatR, CQRS, PostgreSQL y RabbitMQ

1. Presentación

El presente documento describe el proyecto integrador propuesto para el curso, con el propósito de articular en un único problema de negocio los contenidos de fundamentos, arquitectura, diseño orientado al dominio, pruebas, principios SOLID, patrones de diseño, integración continua, contenedores, refactorización y cierre técnico del semestre. El proyecto ha sido diseñado para ser suficientemente acotado en alcance, pero con la riqueza conceptual necesaria para exigir decisiones de arquitectura reales, separación de responsabilidades, modelado del dominio y construcción de funcionalidades en tiempo real.

La propuesta se centra en una plataforma web llamada UMBRAL, destinada a la operación de experiencias narrativas inmersivas tipo investigación interactiva. Este contexto resulta llamativo, poco común y técnicamente fértil, ya que combina gestión administrativa, ejecución operativa, trazabilidad de eventos, cálculo de puntajes, publicación de información en tiempo real y procesamiento asíncrono de eventos del dominio.

2. Nombre del proyecto y descripción general

Nombre del proyecto: UMBRAL.

Descripción general: UMBRAL es una plataforma web para diseñar misiones de investigación inmersiva, crear sesiones en vivo, registrar equipos participantes, liberar pistas, recibir evidencias, aplicar reglas de validación y puntaje, y supervisar toda la operación desde una consola central. El sistema debe permitir que los participantes y los operadores observen cambios de estado en tiempo real y que determinados procesos sean desacoplados mediante colas de mensajería.

3. Enunciado del problema

Una organización dedicada a experiencias inmersivas de misterio y resolución de casos administra actualmente sus sesiones mediante hojas de cálculo, grupos de mensajería y decisiones manuales del operador. Esta dinámica provoca retrasos en la liberación de pistas, dificultades para controlar el avance de varios equipos en paralelo, inconsistencias en los puntajes, falta de trazabilidad y una experiencia heterogénea para los participantes.

La organización requiere una solución centralizada que permita modelar las misiones, controlar la ejecución en vivo de las sesiones, coordinar la participación de múltiples equipos, registrar evidencias, recalcular puntuaciones y visualizar el estado global de la sesión en un tablero de supervisión. La solución debe ser construida como una aplicación web moderna, con actualización en tiempo real, backend desacoplado por responsabilidades y mecanismos de comunicación asíncrona para tareas que no deban bloquear la experiencia principal.

4. Objetivo general

Desarrollar una aplicación web para la operación en tiempo real de experiencias de investigación inmersiva, aplicando principios de arquitectura limpia y hexagonal, modelado del dominio, separación CQRS, persistencia relacional, comunicación en tiempo real y mensajería asíncrona, de forma que la solución permita evidenciar los contenidos teóricos y prácticos del curso dentro de un único proyecto integrador.

5. Objetivos específicos

- Diseñar una arquitectura del sistema que separe con claridad dominio, aplicación, infraestructura e interfaces externas.

- Modelar el problema mediante entidades, agregados, repositorios, value objects y servicios de dominio.
- Implementar una interfaz web para administración, operación y participación en sesiones.
- Construir casos de uso mediante MediatR y patrón CQRS, distinguiendo comandos y consultas.
- Persistir la información del sistema en PostgreSQL mediante Entity Framework Core.
- Incorporar comunicación en tiempo real basada en WebSockets para reflejar cambios instantáneos en la operación.
- Desacoplar procesos secundarios por medio de colas RabbitMQ.
- Aplicar pruebas unitarias, de integración y end-to-end con criterios de cobertura y calidad.
- Demostrar el uso de patrones de diseño y principios SOLID en componentes concretos de la solución.
- Empaquetar la solución para su ejecución en contenedores y automatizar validaciones mínimas mediante integración continua.

6. Alcance del proyecto

El alcance mínimo esperado para la solución comprende los siguientes módulos y capacidades funcionales.

1. Gestión de misiones: creación, edición, consulta y activación de misiones. Cada misión debe incluir nombre, descripción, nivel de dificultad, tiempo máximo, etapas o nodos de investigación, pistas y reglas básicas de avance.
2. Gestión de sesiones en vivo: creación de sesiones a partir de una misión, cambio de estado de la sesión y control del tiempo de ejecución.
3. Gestión de equipos participantes: registro de equipos, asignación a una sesión y consulta de su progreso.
4. Panel del operador: visualización del estado global de la sesión, liberación de pistas, aplicación de penalizaciones y seguimiento del ranking.
5. Panel del equipo: visualización de pistas asignadas, temporizador, puntaje acumulado y envío de respuestas o evidencias.
6. Supervisión en tiempo real: actualización instantánea de ranking, cambios de estado, liberación de pistas y eventos relevantes.
7. Procesamiento asíncrono: publicación y consumo de eventos para auditoría, recálculo de puntajes, alertas y consolidación del historial.
8. Seguridad básica: diferenciación de permisos por rol para administrador, operador y equipo participante.

Quedan expresamente fuera del alcance funcionalidades avanzadas como cobros en línea, integración con dispositivos físicos, módulos complejos de analítica histórica, inteligencia artificial aplicada al contenido de las misiones, geolocalización precisa de participantes o aplicaciones móviles nativas. La propuesta debe concentrarse en una aplicación web bien construida, coherente y técnicamente defendible.

7. Actores del sistema

Actor	Responsabilidades principales	Permisos mínimos esperados

Administrador	Configura misiones, consulta sesiones y mantiene catálogos base.	Crear/editar misión, consultar sesiones, gestionar usuarios.
Operador	Inicia sesiones, libera pistas, aplica penalizaciones y supervisa la ejecución.	Crear sesión, cambiar estado, liberar pista, observar ranking e historial.
Equipo participante	Consulta su tablero, recibe pistas y envía respuestas o evidencias.	Acceder a su sesión, visualizar progreso y registrar evidencias.

8. Requerimientos funcionales

Código	Requerimiento funcional
RF-01	El sistema debe permitir crear, editar, consultar y desactivar misiones.
RF-02	Cada misión debe permitir registrar etapas o nodos, pistas y tiempo máximo de ejecución.
RF-03	El sistema debe permitir crear una sesión en vivo a partir de una misión activa.
RF-04	La sesión debe manejar estados al menos de programada, en preparación, activa, pausada, finalizada y cancelada.
RF-05	El sistema debe permitir registrar equipos participantes y asociarlos a una sesión.
RF-06	Cada equipo debe visualizar su temporizador, puntaje y pistas habilitadas.
RF-07	El operador debe poder liberar pistas de forma manual o condicionada por reglas de avance.
RF-08	Los equipos deben poder enviar respuestas o evidencias asociadas a una etapa de la misión.
RF-09	Cada envío de evidencia debe quedar registrado con fecha, equipo, sesión y estado de validación.
RF-10	El sistema debe recalcular el puntaje del equipo cuando una evidencia sea validada o penalizada.
RF-11	El operador debe poder aplicar penalizaciones justificadas a un equipo.

RF-12	El ranking de la sesión debe mostrarse y actualizarse en tiempo real.
RF-13	El panel del operador debe reflejar cambios de estado y eventos relevantes en tiempo real.
RF-14	La aplicación debe publicar eventos del dominio en RabbitMQ al registrarse evidencias o cambios significativos.
RF-15	Debe existir un historial de eventos de la sesión con trazabilidad mínima para auditoría.
RF-16	La aplicación debe diferenciar funcionalidades por rol.
RF-17	La solución debe permitir consultar misiones, sesiones, equipos y ranking mediante consultas separadas de los comandos.
RF-18	La aplicación debe disponer de validaciones de negocio antes de aceptar cambios de estado o evidencias.

9. Requerimientos no funcionales

Código	Requerimiento no funcional
RNF-01	La solución debe implementarse con frontend en React y backend en .NET Core.
RNF-02	La persistencia principal debe resolverse con PostgreSQL y Entity Framework Core.
RNF-03	La comunicación en tiempo real debe implementarse sobre WebSockets.
RNF-04	La lógica de aplicación debe estructurarse con MediatR y enfoque CQRS.
RNF-05	Los procesos asíncronos deben desacoplarse mediante RabbitMQ.
RNF-06	La solución debe seguir arquitectura hexagonal o una variante compatible con arquitectura limpia.
RNF-07	El dominio no debe depender de infraestructura ni de detalles del framework web.
RNF-08	La aplicación debe incorporar logging, manejo de excepciones y validaciones consistentes.
RNF-09	El backend debe alcanzar como meta académica una cobertura de pruebas de al menos 90 %.
RNF-10	La solución debe poder ejecutarse localmente mediante Docker Compose.

RNF-11	El repositorio debe incluir pipeline de integración continua para compilación y ejecución de pruebas.
RNF-12	La interfaz debe ser clara, utilizable y coherente con los flujos principales del sistema.

10. Reglas de negocio

Código	Regla de negocio
RB-01	Una misión solo puede utilizarse para crear sesiones si se encuentra activa.
RB-02	Una sesión no puede iniciar si no posee al menos un equipo registrado.
RB-03	No se deben aceptar evidencias si la sesión está pausada, finalizada o cancelada.
RB-04	Una pista no puede liberarse dos veces al mismo equipo para la misma etapa.
RB-05	Cada evidencia debe asociarse exactamente a un equipo, una sesión y una etapa.
RB-06	Toda penalización debe registrar motivo y momento de aplicación.
RB-07	El puntaje acumulado de un equipo no puede quedar sin trazabilidad de origen.
RB-08	El ranking debe ordenarse de mayor a menor puntaje y usar tiempo de resolución como criterio de desempate cuando aplique.
RB-09	Los cambios de estado de la sesión deben respetar transiciones válidas.
RB-10	El operador solo podrá administrar sesiones que tenga asignadas o visibles según la política definida por el equipo.

11. Arquitectura y lineamientos técnicos obligatorios

La solución deberá construirse siguiendo una arquitectura que evidencie separación clara de responsabilidades. Se espera una distribución compatible con los siguientes componentes:

- Capa de presentación web en React, separando componentes, vistas, hooks y servicios de acceso a API.
- Backend en .NET Core organizado por dominio, aplicación, infraestructura y capa de entrada.
- Casos de uso implementados mediante MediatR, distinguiendo comandos de escritura y consultas de lectura.
- Persistencia sobre PostgreSQL usando Entity Framework Core y repositorios definidos por contratos.
- Canal de WebSockets para actualizaciones de tablero, ranking, pistas y cambios de estado.
- Cola RabbitMQ para el desacoplamiento de eventos secundarios, tales como auditoría, recálculo y notificaciones.

- Mecanismos de cross-cutting concerns como logging, manejo global de excepciones, validaciones y seguridad básica.
- Empaquetado del sistema para ejecución local mediante Docker Compose.

A nivel de diseño, el proyecto deberá evidenciar el uso de arquitectura hexagonal o una variante equivalente. Los adaptadores primarios pueden estar representados por la API HTTP y los canales de tiempo real, mientras que los adaptadores secundarios estarán representados por la persistencia, la mensajería y cualquier integración externa necesaria para la operación. La propuesta técnica deberá justificar de forma explícita esta separación.

12. Modelo conceptual sugerido del dominio

Concepto	Descripción académica esperada
Mission	Agregado principal para definir una misión, sus etapas, pistas y reglas base.
MissionNode	Entidad o componente jerárquico que representa una etapa, subetapa o pista.
LiveSession	Agregado principal para la ejecución en vivo de una misión.
Team	Entidad asociada a una sesión, con progreso, estado y puntaje.
EvidenceSubmission	Registro de respuesta o evidencia enviada por un equipo.
ScoreEntry	Trazabilidad de puntos otorgados o penalizados.
SessionEvent	Evento significativo del dominio utilizado para historial y auditoría.
Value Objects	Objetos de valor para conceptos como TeamCode, ScoreValue, Difficulty o PenaltyReason.

A efectos académicos, se solicita identificar al menos tres bounded contexts o subáreas con lenguaje propio: diseño de misiones, operación de sesiones y puntuación/monitoreo. No es obligatorio que el equipo implemente microservicios; basta con que evidencie separación conceptual, lenguaje ubicuo y límites claros dentro de la solución.

13. Principios y patrones que deben evidenciarse

El proyecto debe servir como vehículo para aplicar de manera concreta los temas del curso. Por ello, se espera que cada equipo evidencie, en componentes reales del sistema, los principios y patrones que se indican a continuación.

Tema	Evidencia mínima esperada en UMBRAL
Fundamentos y construcción de código	Repositorio Git ordenado, nombres consistentes, manejo básico de excepciones y criterios de robustez.

Arquitectura limpia / hexagonal	Separación explícita de dominio, aplicación e infraestructura; definición de puertos y adaptadores.
DDD	Entidades, agregados, repositorios, servicios de dominio, bounded contexts y lenguaje ubicuo.
Value Objects y dependencias	Uso de objetos de valor e inversión de control con inyección de dependencias.
Pruebas unitarias	Casos de uso, servicios y reglas de negocio cubiertos con pruebas significativas.
Mocks, stubs y fakes	Pruebas desacopladas de infraestructura, especialmente en handlers y servicios de aplicación.
Cobertura	Reporte de cobertura que evidencie la meta académica establecida.
SOLID	Justificación de SRP, OCP, LSP, ISP y DIP en clases reales del proyecto.
Cross-cutting concerns	Logging, middleware de excepciones, validaciones y seguridad básica por roles.
Strategy	Estrategia de cálculo de puntaje según dificultad, modo de sesión u otra variación del dominio.
Composite	Representación jerárquica de misión, etapas y subetapas.
Facade	Servicio coordinador para operación de sesión o publicación de eventos.
Proxy	Control de acceso a recursos sensibles como pistas o paneles restringidos.
Template Method	Flujo base de procesamiento de evidencias con pasos definidos y variantes.
State	Gestión del ciclo de vida de la sesión mediante estados válidos.
Chain of Responsibility	Cadena de validaciones para aceptar o rechazar evidencias o cambios de estado.
Integración continua	Pipeline con restauración, compilación, pruebas y reporte de resultado.
Docker y Docker Compose	Ejecución local del frontend, backend, base de datos y mensajería mediante contenedores.
Pruebas de integración y E2E	Pruebas que verifiquen persistencia, endpoints y flujos principales de uso.
Refactorización	Registro de mejoras estructurales aplicadas al proyecto durante el semestre.

14. Entregables académicos sugeridos

Hito	Contenido mínimo esperado
Proyecto 1	Esqueleto funcional con arquitectura base, modelo de dominio inicial, repositorio Git, persistencia base, primeros casos de uso y pruebas unitarias iniciales.
Avance intermedio	Aplicación de patrones de diseño seleccionados, WebSockets operativos, integración con RabbitMQ, pipeline básico y evidencia de principios SOLID.
Proyecto 2 / entrega final	Aplicación completa, contenedorizada, con pruebas unitarias, integración y E2E, cobertura objetivo, documentación técnica y presentación funcional.

Cada equipo deberá acompañar los entregables con una breve memoria técnica que describa las decisiones de arquitectura, el modelo del dominio, los patrones implementados, las pruebas realizadas, los hallazgos de refactorización y las instrucciones para levantar la solución en ambiente local.

15. Criterios mínimos de aceptación

- La solución compila, ejecuta y permite levantar sus componentes principales sin intervención manual excesiva.
- Existe evidencia clara de separación arquitectónica y aplicación de CQRS con MediatR.
- El modelo del dominio es coherente y utiliza al menos los principales elementos tácticos vistos en clase.
- Las funcionalidades críticas del flujo de sesión en vivo operan correctamente.
- La actualización en tiempo real se demuestra de manera observable.
- La publicación y consumo de mensajes por RabbitMQ se demuestra con al menos un flujo de negocio relevante.
- El proyecto incluye pruebas y reportes técnicos mínimos sobre cobertura y calidad.
- La documentación técnica permite comprender la solución y reproducir el entorno local.

16. Criterios de evaluación

Dimensión	Ponderación sugerida	Aspectos a observar
Arquitectura y modelado del dominio	20 %	Coherencia de capas, bounded contexts, agregados, repositorios y decisiones de diseño.
Cumplimiento funcional	25 %	Cobertura del alcance, consistencia del flujo principal y estabilidad de la solución.
Calidad del código y aplicación de principios	20 %	SOLID, patrones, legibilidad, mantenibilidad y manejo de errores.

Pruebas y aseguramiento de calidad	15 %	Pruebas unitarias, integración, E2E y evidencia de cobertura.
Infraestructura y despliegue local	10 %	Pipeline, Docker Compose, configuración y reproducibilidad.
Documentación y presentación	10 %	Claridad expositiva, defensa técnica y calidad de la memoria final.

17. Consideraciones para la presentación final

La defensa del proyecto deberá centrarse en el valor de la solución, las decisiones de arquitectura, el modelo del dominio, la demostración del flujo principal en vivo y la evidencia objetiva de los temas académicos cubiertos. No se espera una presentación meramente comercial; se espera una exposición técnica y argumentada, donde el equipo pueda justificar por qué su diseño responde al problema planteado y cómo cada componente implementado aporta a la calidad de la solución.

- Demostración del flujo completo: creación de sesión, registro de equipo, liberación de pista, envío de evidencia y actualización del ranking.
- Explicación del modelo del dominio y de los patrones implementados.
- Demostración del uso de WebSockets y RabbitMQ dentro del flujo del negocio.
- Presentación de pruebas, cobertura, pipeline y contenedores.
- Breve recuento de problemas encontrados y refactorizaciones realizadas.

18. Cierre

UMBRAL constituye una propuesta académica equilibrada: evita caer en dominios excesivamente comunes, mantiene un alcance razonable para un semestre y, al mismo tiempo, ofrece suficientes puntos de entrada para poner en práctica todos los temas planteados en el plan de clases. Su naturaleza operativa, narrativa y en tiempo real obliga a los equipos a diseñar con criterio, probar con rigor y construir una solución técnicamente defendible, que pueda ser evaluada desde la calidad del código, la arquitectura, la experiencia funcional y la solidez de la entrega final.